ANCONnet

Documentación técnica del sistema

Intranet corporativa · Grupo Ancon

Revisión: 21/10/2026

ANCONnet — Documentación técnica

> Intranet corporativa de Grupo Ancon (frontend `newAnconnet`). Última revisión: 21/10/2026 · Estado del código: rama `main` (scaffold + trabajo sin commitear).

Este documento describe la arquitectura, los módulos, la seguridad, las validaciones, la estructura de carpetas y las convenciones del proyecto, con el objetivo de que cualquier desarrollador pueda entender el sistema completo, mantenerlo y extenderlo.

Índice

1. Visión general
2. Stack tecnológico
3. Arquitectura
4. Estructura de carpetas
5. Configuración y entorno
6. Autenticación y sesión
7. Autorización (RBAC)
8. Base de datos (Prisma)
9. Capa de acceso al backend (`lib/api`)
10. Módulos funcionales
11. Validaciones
12. Componentes compartidos y UI
13. Infraestructura transversal
14. Generación de documentos (PDF / Word / CSV)
15. Testing y calidad
16. Convenciones de código
17. Guías: cómo extender el sistema
18. Despliegue
19. Estado actual, deuda técnica y riesgos
20. Glosario

---

1. Visión general

ANCONnet es la intranet que reemplaza al ERP inmobiliario legacy de Grupo Ancon (Inmoconstrucciones). Cubre el ciclo comercial completo de venta de lotes y la operación interna:

Área	Qué resuelve
Ventas	Inventario de lotes, proceso de venta (cliente → documentos → oferta → financiamiento → pago → licitud/convenio), ventas urgentes, reportes de ventas
Gestión	Cotizaciones, refinanciamientos, entregas de lotes
Tesorería	Revisión/aprobación de depósitos de clientes (tiempo real)
RRHH	Vacaciones, permisos, préstamos, expedientes y documentos
Reportes	Reportes programados + constructor de reportes dinámicos
App móvil	Administración de la app de clientes: usuarios, notificaciones push, diagnóstico/telemetría
Administración	Usuarios, roles y permisos, auditoría, convenios (plantillas legales), promociones

Modelo de despliegue lógico:

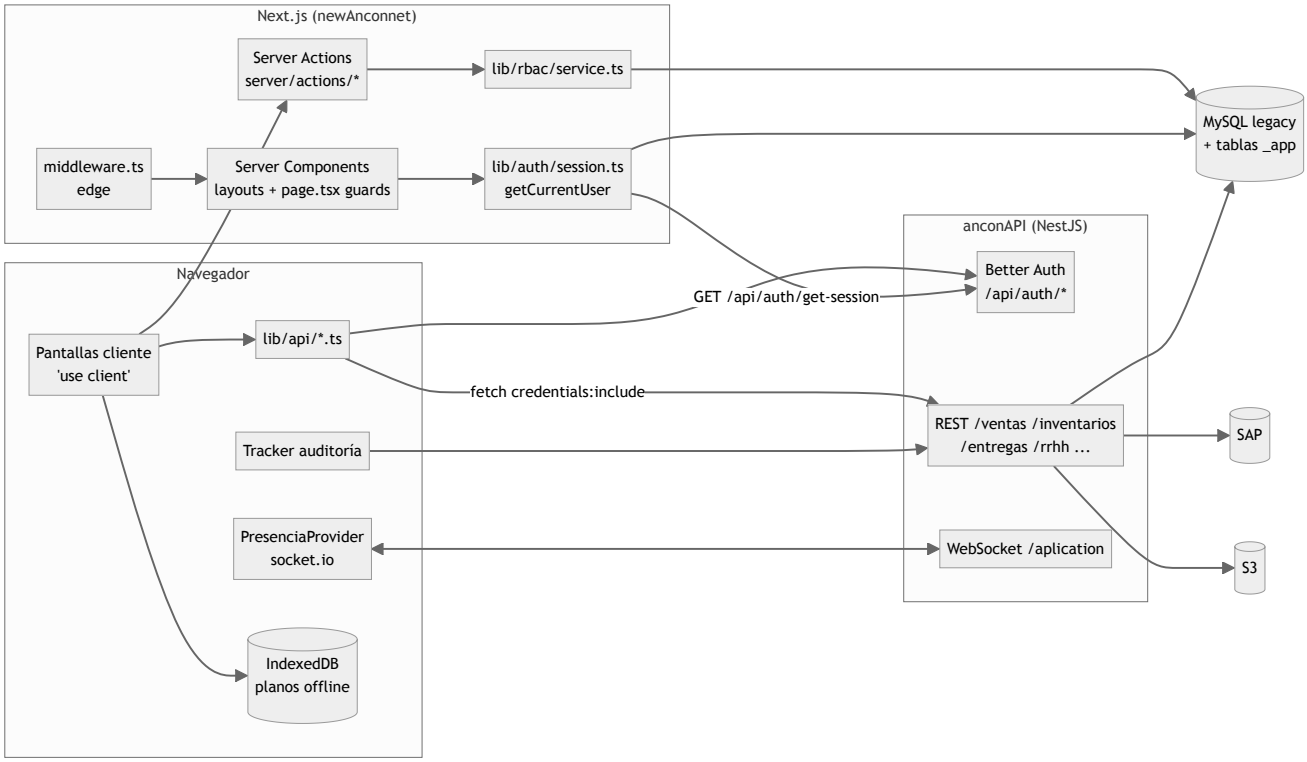
- `newAnconnet` (este repo): Next.js 14. Renderiza la UI, resuelve la sesión del usuario y gestiona el RBAC (escribe directamente en MySQL vía Prisma).
- `anconAPI` (repo separado, NestJS): backend de negocio. Expone REST + WebSocket, maneja Better Auth (login + 2FA), SAP, S3, notificaciones push, auditoría, etc.
- **MySQL compartido**: la base de datos legacy real, a la que acceden ambos.

2. Stack tecnológico

Capa	Tecnología	Versión	Notas
Framework	Next.js (App Router)	14.2.35	`next dev -H 0.0.0.0` para acceso desde la LAN
Lenguaje	TypeScript	5	`strict: true`, alias `@/*` → raíz
UI	React	18	Requerido por Next 14
Estilos	Tailwind CSS v4 + `@tailwindcss/postcss`	4.3	Tokens en `app/globals.css`
Componentes	shadcn/ui (estilo `radix-nova`) sobre `radix-ui`	—	Primitivos en `components/ui/`
Iconos	lucide-react	—	
Formularios	react-hook-form + zod + @hookform/resolvers	zod 4	Mismo schema cliente/servidor
ORM	Prisma	6.19	Fijado en 6.x (7.x cambia a `prisma.config.ts` ESM-only)
BD	MySQL	—	Base legacy compartida
Auth propia	argon2id (`argon2`) + JWT HS256 (`jose`)	—	Sistema legacy de esta app
Auth real	Better Auth (en anconAPI)	**1.6.11 fijo**	Ver §6
Tiempo real	socket.io-client	4.8	Namespace `/aplication`
Editor rico	Tiptap 2 (+ tablas, color, fuente, alineación)	2.27	Plantillas de convenios
Documentos	jsPDF, docx, html-to-docx, jszip, pdfjs-dist	—	Ver §14
Tests	Vitest + Testing Library + jsdom	4.1	
Calidad	ESLint (next + prettier), Prettier + plugin Tailwind	—	

3. Arquitectura

3.1 Diagrama de componentes



3.2 Principios de diseño

1. **El navegador habla directo con anconAPI.** Las pantallas son mayoritariamente componentes cliente que llaman a `lib/api/*.ts` con `credentials: "include"` (cookie de Better Auth). Next.js **no** actúa como proxy de datos de negocio.
2. **Next.js resuelve identidad y permisos en servidor.** Cada `page.tsx` protegida es un Server Component que llama a un guard (`requirePermiso`, etc.) antes de renderizar el componente cliente `*-content.tsx`.
3. **Defensa en profundidad:** middleware (filtro rápido) → layout `(dashboard)` (`getCurrentUser`) → guard de página → `<Can>` en UI (solo UX) → **validación real en anconAPI** (fuente de verdad de la autorización de cada operación).
4. **RBAC compartido:** las tablas `*_app` las escribe esta app (panel admin) y las leen tanto esta app como el guard de anconAPI (`lib/permisos-rbac.ts` del backend).
5. **Las tablas legacy no se alteran** desde este repo (solo lectura), salvo `user.permissions` (recompilado por compatibilidad, ver §7.5).

3.3 Patrón de página

```
app/(dashboard)/<modulo>/page.tsx          ← Server Component: guard + metadata
app/(dashboard)/<modulo>/<x>-content.tsx   ← "use client": estado, fetch vía lib/api
app/(dashboard)/<modulo>/<x>-sheet.tsx     ← paneles laterales (Sheet) de crear/editar/detalle
app/(dashboard)/<modulo>/datos.ts|formato.ts|*-utils.ts  ← tipos, catálogos y helpers puros
```

Ejemplo:

```
// app/(dashboard)/ventas/urgentes/page.tsx
export default async function Page() {
  await requirePermiso("venta:urgente", "Ventas urgentes")
  return <VentasUrgentesContent />
}
```

4. Estructura de carpetas

```
newAnconnet/
├── app/                                Rutas (App Router)
│   ├── layout.tsx                      Raíz: fuentes, metadata PWA, ToastProvider, AuthProvider
│   ├── globals.css                     Tokens de diseño (claro/oscuro) + Tailwind v4
│   ├── manifest.ts                     /manifest.webmanifest (PWA instalable)
│   ├── not-found.tsx                   404
│   ├── sin-acceso/page.tsx             Pantalla "No tienes acceso a <sección>"
│   ├── api/auth/clear-session/         Route handler: borra cookies y va a /login
│   ├── (auth)/                         Grupo público
│   │   ├── login/                      Email + contraseña (Better Auth)
│   │   └── verificar/                  Código OTP (2FA)
│   └── (dashboard)/                    Grupo autenticado (layout con AppShell)
│       ├── page.tsx                    Inicio: KPIs, gráfico, tareas, calendario, entregas
│       ├── admin/                      usuarios · roles · auditoria
│       ├── app/                        usuarios · notificaciones · diagnostico (app móvil)
│       ├── configuracion/              convenios (+representantes) · promociones
│       ├── entregas/                   listado · nuevo · [id]/pre-entrega|entrega|cierre
│       ├── gestion/                    cotizaciones · refinanciamiento · encuesta
│       ├── notificaciones/             (versión anterior del envío push; fuera del menú)
│       ├── proyectos/                  Selector de proyecto de "Nueva Venta"
│       ├── reportes/                   Reportes + administrar (constructor)
│       ├── rrhh/                       Tabs: vacaciones, permisos, préstamos, usuarios,
documentos
│       ├── tesoreria/                  depositos · retiros (maqueta)
│       └── ventas/                     inventario · lotes · nuevo-proceso · procesos · oferta ·
│                                       financiamiento · pagos · licitud · convenio · mis-ventas
.
│                                       realizadas · reporte · revision-documentos · urgentes
├── components/
│   ├── ui/                             Primitivos shadcn (Button, Sheet, Select, Tabs, ...)
│   ├── shared/                         Compuestos reutilizables (AppShell, SidebarNav, Can,
...)
│   └── convenios/                      Editor Tiptap, gestor de variables, asignaciones
├── lib/
│   ├── api/                            Un cliente HTTP por prefijo de backend (ver §9)
│   ├── auth/                           config, jwt, password, rate-limit, rbac, session, guard
│   ├── rbac/service.ts                 Gestión RBAC + permisos efectivos (server-only)
│   ├── auditoria/tracker.ts            Cola de eventos de auditoría del navegador
│   ├── db/client.ts                    PrismaClient singleton
│   ├── validations/                    Schemas zod y validadores puros
│   ├── const/                          Catálogos: actividades, nacionalidades, ubicación,
│   │                                   códigos telefónicos, documentos de proceso
│   ├── reportes/catalogo.ts            Catálogo de reportes programados
│   ├── entregas-reporte.ts             PDF/CSV del listado de entregas
│   ├── rrhh-reportes-pdf.ts            PDFs de RRHH (préstamo, permiso, vacaciones)
│   ├── importar-lotes.ts               Parser CSV / pegado de Excel
│   ├── plano-cache.ts                  Cache offline de planos (IndexedDB)
│   ├── pdf-a-imagen.ts                 1ª página de PDF → PNG (pdfjs)
│   ├── limpiar-html-word.ts            Sanea HTML pegado desde Word
│   ├── navigation.ts                   Re-export de next/navigation con barra de progreso
│   ├── navigation-progress.ts          Estado de la barra de progreso
│   └── utils.ts                        cn() (clsx + tailwind-merge)
├── server/actions/                     Server Actions: auth/login, auth/logout, rbac
```

```
├── prisma/
│   ├── schema.prisma                Tablas legacy (introspectadas) + tablas *_app
│   ├── seed.ts                      Catálogo inicial de roles/permisos
│   └── migrar-catalogo-permisos.ts  Pobla permisos_app desde user.permissions
├── types/                           auth.ts (AuthenticatedUser), html-to-docx.d.ts
├── tests/                           Vitest (espeja lib/ y app/)
├── public/                          sw.js, offline.html, íconos, imágenes de proyectos
├── informes/                        PDFs: catálogo de permisos, informe de seguridad
├── certificates/                    Certificados locales para `start:dev` (HTTPS)
├── middleware.ts                    Filtro de rutas en edge
├── next.config.mjs                  Fallbacks webpack (fs/path/encoding) para html-to-docx
├── components.json                  Configuración shadcn
└── vitest.config.ts · tsconfig.json · .eslintrc.json · .prettierrc.json
```

5. Configuración y entorno

5.1 Variables de entorno (`.env`)

Variable	Obligatoria	Uso
`DATABASE_URL`	Sí	MySQL (`mysql://usuario:password@host:puerto/bd`). **Ojo:** `.env.example` muestra un ejemplo PostgreSQL desactualizado; el `provider` real es `mysql`.
`AUTH_JWT_SECRET`	Sí (≥ 32 caracteres)	Firma del JWT de la sesión legacy. Si falta o es corto, `getJwtSecretKey()` lanza error (y el middleware también lo usa).
`AUTH_SESSION_TTL_SECONDS`	No (default 28800 = 8 h)	Duración de la sesión legacy.
`NEXT_PUBLIC_API_URL`	Sí	URL de anconAPI. **Único lugar** a cambiar para apuntar a otro backend (todos los `lib/api/*`, el socket y `session.ts` la leen; fallback `http://localhost:3001`).

5.2 Scripts

Comando	Qué hace
`npm run dev`	Servidor de desarrollo en `0.0.0.0:3000` (`.claude/launch.json` usa puerto 3002)
`npm run start:dev`	Igual con `--experimental-https` (usa `certificates/`) — necesario para probar PWA/cámara desde otro dispositivo
`npm run build` / `start`	Build y servidor de producción (el build falla con errores de tipos o lint)
`npm run lint` / `format` / `format:check`	ESLint / Prettier
`npm run test` / `test:watch`	Vitest
`npm run db:generate`	`prisma generate`
`npm run db:seed`	Catálogo de roles/permisos
`npm run db:studio`	Prisma Studio
`npx tsx prisma/migrar-catalogo-permisos.ts`	Migración idempotente del catálogo de permisos

> `db:migrate` existe en `package.json` pero **no debe usarse** contra la BD real (ver §8.3).

5.3 Puesta en marcha local

```
npm install
cp .env.example .env        # completar DATABASE_URL (MySQL), AUTH_JWT_SECRET,
NEXT_PUBLIC_API_URL
npm run db:generate
npm run db:seed             # opcional: catálogo base de roles/permisos
npm run dev
```

Requisitos: Node.js 20+, acceso a la BD MySQL de trabajo y a una instancia de anconAPI (sin anconAPI no se puede iniciar sesión).

---

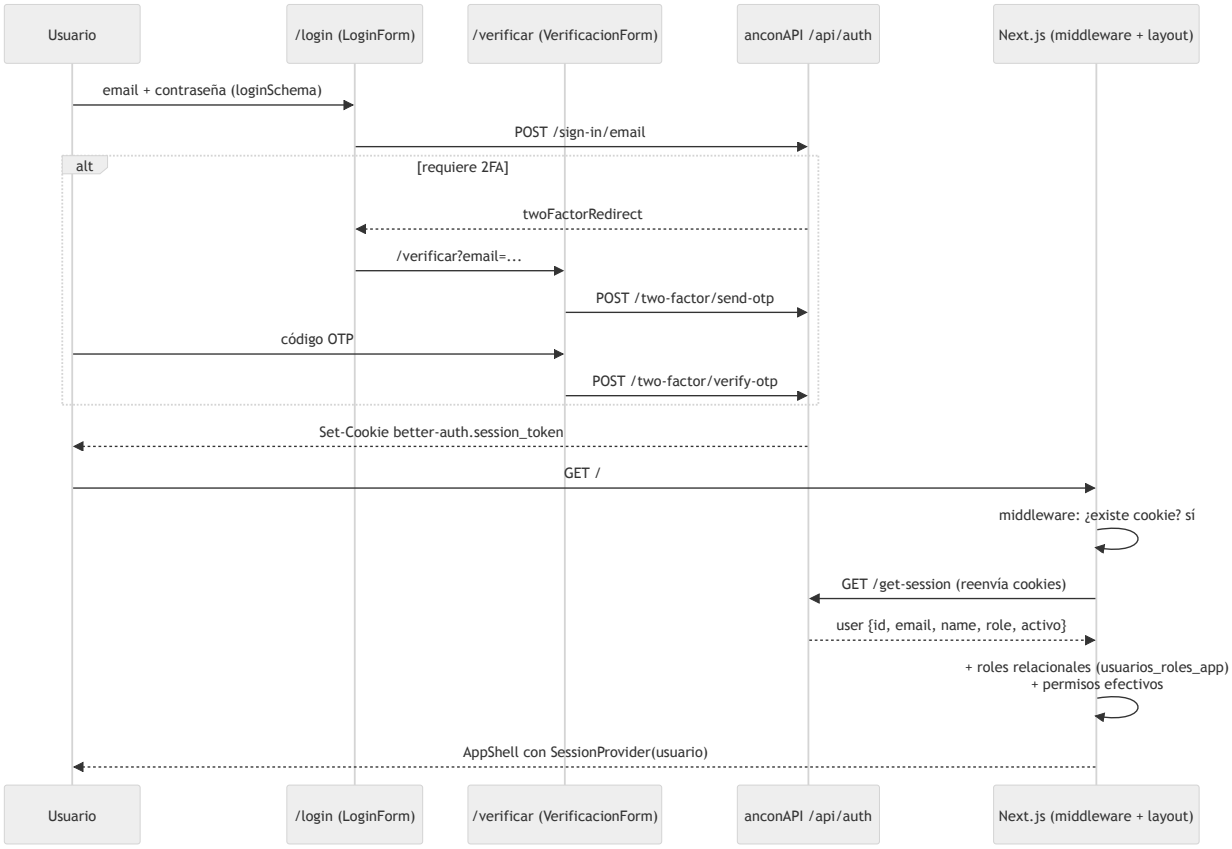
6. Autenticación y sesión

6.1 Dos sistemas conviviendo

	Sesión Better Auth (actual)	Sesión legacy propia
Dónde se autentica	anconAPI `/api/auth/*`	Server Action `server/actions/auth/login.ts`
Cookie	`better-auth.session_token` (opaca)	`session` (JWT HS256, httpOnly, Secure, SameSite=Strict)
Password	`account.password` (Better Auth)	`credenciales_acceso.passwordHash` (argon2id)
Registro de sesión	tabla `session`	tabla `sesiones_app` (por `jti`, revocable)
2FA	Sí (OTP por correo)	No
Estado	**Es la que usa el formulario de login**	Código presente pero el formulario actual no la invoca

`getCurrentUser()` (`lib/auth/session.ts`) es la **única fuente de verdad** de "quién es el usuario": prueba primero la sesión legacy (sin red) y, si no hay, valida la de Better Auth.

6.2 Flujo de login (Better Auth + 2FA)



6.3 Construcción del usuario autenticado (`AuthenticatedUser`)

```
type AuthenticatedUser = {
  id: string; email: string; nombre: string
  roles: string[]                                  // nombres de rol
  permisos: { recurso: string; accion: string }[]  // permisos efectivos
}
```

Para la sesión Better Auth:

1. `GET {API}/api/auth/get-session` con las cookies del request (`cache: "no-store"`).
2. Si `user.activo === false` → sin sesión.
3. `roles` = `"Administrador"` si `user.role === "admin"` + nombres de `usuarios_roles_app`. Si queda vacío → `["Colaborador"]`.
4. `permisos` = `getPermisosEfectivosDeUsuario(id)` (§7.3).

6.4 Middleware (`middleware.ts`)

- Rutas públicas: `/login`, `/verificar`. Si ya hay sesión, redirige a `/`.
- Sin cookie (Better Auth ni JWT válido) → `/login?from=<ruta>`.
- Filtro de rol por prefijo: `/admin` → `Administrador` (solo aplicable a la sesión legacy, porque la cookie de Better Auth es opaca en edge).
- El `matcher` excluye `api`, estáticos, `manifest.webmanifest`, `sw.js`, `offline.html`, `apple-touch-icon.png` e `icons/` (necesario para que la PWA sea instalable).
- Las URLs de redirección se arman desde el header `Host` (con `-H 0.0.0.0`, `request.url` puede resolver a `0.0.0.0`).

6.5 Cierre / invalidación de sesión

- **`/api/auth/clear-session`**: borra ambas cookies y redirige a `/login?from=`. Se usa cuando un Server Component detecta sesión inválida con cookie presente (un RSC no puede borrar cookies; ir directo a `/login` provocaría un loop con el middleware).
- **`SesionGuardian`** (cliente): intercepta `window.fetch`; ante un `401` de anconAPI (excepto `/api/auth/*`) confirma con `get-session` y, si realmente expiró, navega a `clear-session`.
- **Logout**: `signOut()` de Better Auth + Server Action `logout()` (revoca `sesiones_app`).
- Admin → Usuarios permite listar y revocar sesiones de cualquier usuario.

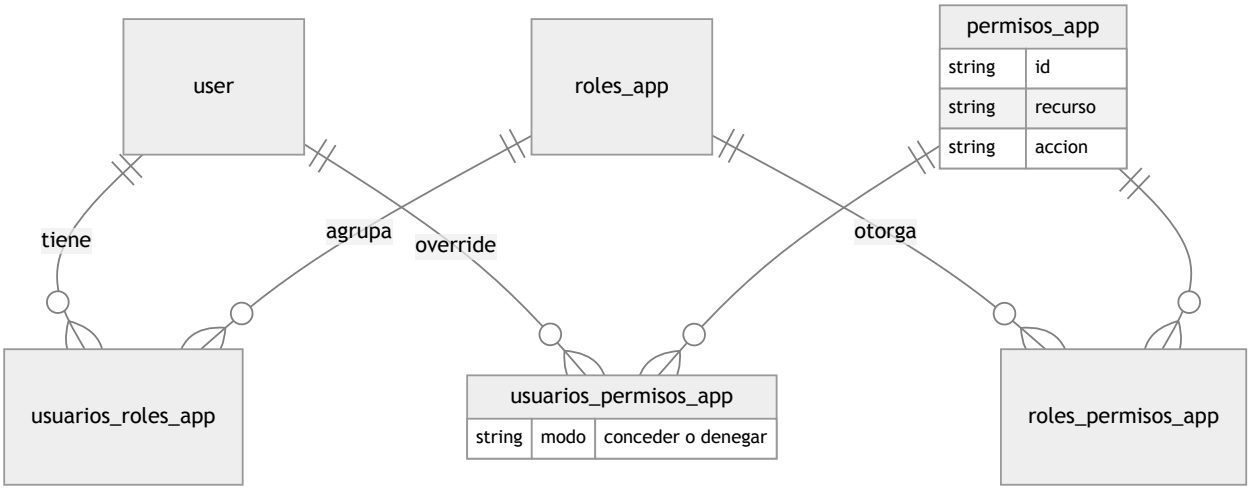
6.6 Rate limiting (sesión legacy)

`lib/auth/rate-limit.ts`: 10 intentos fallidos / 15 min por `IP:email`, **en memoria del proceso**. Con más de una instancia hay que reemplazarlo por Redis/Upstash manteniendo la interfaz `isRateLimited / recordFailedAttempt / resetAttempts`. El rate limit del login real lo aplica Better Auth en anconAPI.

---

7. Autorización (RBAC)

7.1 Modelo



- **Permiso** = `recurso:accion` (único). Ej: `ventas:crear`, `entregas:pre_entrega`.
- `recurso:all` cubre cualquier acción del recurso.
- **Rol** = conjunto de permisos. Un usuario puede tener varios roles.
- **Override por usuario**: `conceder` (suma aunque ningún rol lo dé) o `denegar` (resta aunque un rol lo dé; **denegar siempre gana**).

7.2 Funciones de verificación (`lib/auth/rbac.ts`)

Función	Bypass Administrador	Uso
`can(usuario, recurso, accion, ctx?)`	**Sí**	Caso general
`tienePermisoExplicito(usuario, recurso, accion)`	**No**	Módulos cuyo backend valida sin bypass (Entregas, Reportes), para no mostrar pantallas que luego fallan con 403
`assertCan(...)`	Sí	Lanza `PermisoDenegadoError` (server actions / route handlers)

Separación de funciones: `can(u, x, "aprobar", { creadoPorId })` devuelve `false` si `creadoPorId === u.id`, incluso para administradores.

7.3 Permisos efectivos

```
efectivos = ∪ permisos(roles del usuario) + overrides "conceder" − overrides "denegar"
```

Calculados **en vivo** por `getPermisosEfectivosDeUsuario()` (`lib/rbac/service.ts`) en cada request que resuelve la sesión. `contarPermisosEfectivosPorUsuario()` es la versión batched para listados.

7.4 Dónde se aplica

Capa	Mecanismo
Edge	`middleware.ts` (solo cookie / rol legacy)
Layout	`app/(dashboard)/layout.tsx` → `getCurrentUser()`
Página	`lib/auth/guard.ts`: `requirePermiso`, `requirePermisoExplicito`, `requireAlgunPermiso`, `requireAlgunPermisoExplicito`, `requireUsuario` → redirige a `/login` o `/sin-acceso?seccion=`
Menú	`SidebarNav`: `rolesPermitidos`, `permiso`, `permisoExplicito`, `permisosAlguno`; oculta grupos sin hijos visibles
Componentes	`<Can permiso="x:y" explicito?>` y `useCan()` — **solo UX**
Server Actions RBAC	`server/actions/rbac.ts` → `assertAdmin()` (rol Administrador)
Backend	Guard de anconAPI contra las mismas tablas — **autoridad final**

7.5 Gestión (panel admin)

`server/actions/rbac.ts` expone: listar roles/permisos, crear/renombrar/eliminar rol, asignar permisos a rol, crear/eliminar permiso, asignar roles a usuario, overrides y conteo de permisos. Cada cambio llama a `recompilarPermisosUsuario()`, que **además escribe** `user.permissions` (string CSV) por compatibilidad; ese campo ya no se usa para autorizar.

7.6 Catálogo de permisos usados en el código

Recurso	Acciones	Dónde
`admin`	`user`	Admin → Usuarios / Roles
`auditoria`	`ver`	Admin → Auditoría
`app`	`diagnostico`, `usuarios`, `notificaciones`	App móvil
`dashboard`	`all`	Inicio
`eventos`	`gestionar`	Calendario del inicio
`ventas`	`nueva`, `crear`, `reporte`	Nueva venta, procesos, reportes
`venta`	`urgente`	Ventas urgentes
`inventario`	`ver`, `gestion`, `crear`, `editar`, `eliminar`	Catálogo / gestión de inventario
`proyectos`	`gestion`	Editor de planos
`documentos`	`revisar`	Revisión de documentos
`ofertas`	`editar`, `revisar`	Oferta de compra
`financiamiento`	`editar`, `aprobar`	Financiamiento
`pago`	`editar`, `aprobar`	Pagos
`licitud`	`imprimir`	Reporte de licitud de fondos

Recurso	Acciones	Dónde
`convenios`	`gestionar`, `imprimir`	Plantillas / reporte de convenio
`promociones`	`gestionar`	Promociones
`cotizaciones`	`ver_todas`	Pestaña "Todas" del cotizador
`refinanciamiento`	`crear`, `editar`, `aprobar`	Refinanciamientos
`entregas`	`ver`, `crear`, `pre_entrega`, `reagendar`, `entregar`, `cerrar`, `reporte`	Entregas (explícitos)
`depositos`	`ver`, `gestionar`	Tesorería
`rrhh`	`gestionar`	RRHH (administración)
`reportes`	`ventas`, `administrativo`, `depositos`, `administrar`, `<slug>`	Reportes (explícitos)

El catálogo completo vigente vive en `permisos_app` (ver también `informes/catalogo-permisos-anconnet.pdf`).

8. Base de datos (Prisma)

8.1 Organización de `prisma/schema.prisma`

1. **Tablas legacy** (~47, introspectadas con `prisma db pull`). No se alteran. Grupos principales:

Dominio	Tablas
Identidad (Better Auth)	`user`, `session`, `account`, `verification`, `twoFactor`
Proceso de venta	`sale_process`, `ofertas`, `ofertaMama`, `finan_cabec`, `finan_det`, `sale_paymnt`, `double_lots`, `urgentSale`, `encuesta_calidad`
Refinanciamiento	`refinancian_cabec`, `refinancian_detall`
Cotizador	`cotizador`, `cot_cabec`, `cot_det`
Inventario	`inventario`, `inventario_arch`, `inventario_mod`, `proyecto` (`@@ignore`: sin PK)
Entregas	`entr_process`, `entr_reagend`, `lote_pre_entr`, `lote_entr`
Tesorería	`d_depositos`, `depositos_comprobante`, `retiros`, `retiros_*`, `res_clientes`, `ctbccc`
Comisiones	`acomisiones_*`, `aperiodos_comisiones`, `agg_finan_user`, `advisor_extra_lots`, `excluded_advisor`
RRHH	`holidays`, `prestamos`, `permissions` (solicitudes de permiso)
Otros	`activity_logs`

> Muchas tablas de negocio nuevas (p. ej. `oferta_pld`, `oferta_licitud_fondos`, documentos de proceso, convenios, promociones, auditoría, telemetría) las gestiona **anconAPI** y no están modeladas aquí: este repo no las consulta directamente.

2. **Tablas nuevas de esta app** (100 % aditivas, `@@map` con sufijo `_app`):

Modelo	Tabla	Propósito
`CredencialAcceso`	`credenciales_acceso`	Password argon2id propio + token de reset (1:1 con `user`)
`Sesion`	`sesiones_app`	Sesiones legacy revocables por `jti`
`Rol`	`roles_app`	Roles
`UsuarioRol`	`usuarios_roles_app`	user ↔ rol
`Permiso`	`permisos_app`	`recurso` + `accion` (único)
`RolPermiso`	`roles_permisos_app`	rol ↔ permiso
`UsuarioPermiso`	`usuarios_permisos_app`	overrides `conceder` / `denegar`

8.2 Columnas de `user` relevantes

`id`, `name`, `email`, `role` (`admin` → Administrador), `banned`, `twoFactorEnabled`, `activo`, `permissions` (legacy, no autoriza), y datos de RRHH: `fechaIngreso`, `diasVacaciones`, `diasAumentados`,

`diasDescontados`, `anosTrabajados`, `cargo`, `departamento`, `empresa`.

8.3 Por qué NO usar `prisma migrate`

La BD tiene datos reales y configuraciones (`'0000-00-00'`, colaciones mixtas) incompatibles con la shadow database de `migrate dev`. Procedimiento para cambiar tablas `_app`:

1. Editar `schema.prisma`.
2. `npx prisma migrate diff --from-schema-datasource prisma/schema.prisma --to-schema-datamodel prisma/schema.prisma --script > cambio.sql`
3. **Revisar** que solo haya `CREATE TABLE` / `ALTER` sobre tablas `_app` (nunca sobre legacy).
4. Aplicar a mano y correr `npm run db:generate`.

Los `usuarioId` usan `VARCHAR(36)` con collation `utf8mb4_0900_ai_ci` (igual que `user.id`); sin esto falla la FK.

8.4 Cliente

`lib/db/client.ts`: singleton en `globalThis` para no multiplicar conexiones con el hot reload. Logs: `warn` + `error` en desarrollo, `error` en producción. Prisma solo se usa en servidor (`server-only`).

---

9. Capa de acceso al backend (`lib/api`)

Regla: ninguna pantalla hace `fetch` directo. Hay un archivo por **prefijo de ruta** del backend (detalle y plantilla en `lib/api/README.md`).

9.1 Anatomía de un cliente

1. Constante `BASE_URL = NEXT_PUBLIC_API_URL + prefijo`.
2. Clase de error propia (`XxxApiError` con `status` y `code`).
3. Wrapper privado `xxxFetch<T>()`: `credentials: "include"`, `Content-Type: application/json`, parsea JSON, lanza el error con `message` / `error` del cuerpo; `status 0` = sin conexión.
4. Funciones exportadas tipadas, una por endpoint, con comentario `// VERBO /ruta`.

9.2 Inventario de clientes

Archivo	Prefijo	Endpoints principales
`auth-api.ts`	`/api/auth`	`sign-in/email`, `sign-up/email`, `two-factor/send-otp`, `two-factor/verify-otp`, `sign-out`, `get-session`
`admin-api.ts`	`/api/auth/admin`	`list-users`, `create-user`, `update-user`, `set-role`, `ban-user`, `unban-user`, `remove-user`, `set-user-password`, `list-user-sessions`, `revoke-user-session(s)`
`ventas-api.ts`	`/ventas`	Procesos (`validar/:lote`, `procesos`, `procesos/lote/:lote`, `procesos/:id`, `venta-historica`), lotes libres/vendidos, documentos de proceso (subir, renombrar, eliminar, solicitar, verificar, pendientes de revisión), oferta (CRUD, financiero, licitud-fondos, observaciones, aprobar, devolver), clientes extra, financiamiento (crear, regenerar, cuotas, aprobar), refinanciamientos (crear, regenerar, cuotas, aprobar, rechazar, historial), cotizaciones (CRUD, regenerar, cuotas, promociones), pago (CRUD, aprobar, devolver), promociones, tipos de inmueble, ventas urgentes, `documento-identidad` (OCR), asesores, representantes, `mis-ventas`, `reporte-ventas`
`convenios-api.ts`	`/ventas`	`convenios` (CRUD, asignaciones, preview), `catalogo-variables`, `convenio-representantes`, `procesos/:id/convenio`
`inventario-api.ts`	`/inventarios`	Lotes (CRUD, `importar`, `:id/estado`, posición en plano), proyectos (CRUD, `orden`, `plano`, `imagen`)
`entregas-api.ts`	`/entregas`	Listado, asesores, `sap/:lote`, crear, detalle, pre-entrega, reagendar, entrega (multipart), conflictos de agenda, completar paso, cerrar
`depositos-api.ts`	`/depositoss`	Listar, `upload-recibo`, `aproved`, `reject`, `changeState` (nombres reales del backend)
`rrhh-api.ts`	`/rrhh`	`mi-perfil`, permisos, préstamos, colaboradores/saldo, periodos y solicitudes de vacaciones, vacaciones colectivas, documentos (URLs firmadas)
`reportes-api.ts`	`/reportes`	`administrativo/ventas-por-proyecto`, `mis-reportes`, `definiciones/:slug/datos`, `admin/datasets`, `admin/definiciones`
`auditoria-api.ts`	`/auditoria`	`eventos` (envío desde web), `logs`, `logs/:id`, `catalogos`

Archivo	Prefijo	Endpoints principales
`telemetria-api.ts`	`/telemetry`	`summary`, `trends`, `errors`, `events`, `events/:id`, `sessions/:session`, usuarios recientes
`notificaciones-api.ts`	`/notifications`	Crear, historial, detalle, entrega, editar, cancelar, `send-now`
`notificaciones-internas-api.ts`	`/notificaciones-internas`	Mis notificaciones, no leídas, marcar leída(s)
`dashboard-api.ts`	`/dashboard`	`resumen` (KPIs + ventas por mes)
`tareas-api.ts`	`/tareas`	Listar, crear, `:id/toggle`, eliminar
`eventos-api.ts`	`/eventos`	Listar por mes, crear, eliminar
`saps-api.ts`	`/saps`	`findOneOrderByLote/:lote`

9.3 Patrón de consumo

```
useEffect(() => {
  let cancelado = false
  listarAlgo()
    .then((d) => { if (!cancelado) setItems(d) })
    .catch((e) => { if (!cancelado) setError(e instanceof AlgoApiError ? e.message : "No se
pudo cargar") })
  return () => { cancelado = true }
}, [])
```

---

10. Módulos funcionales

> Convención de estados que se repite: **pendiente** (sin crear) → **ingresado** (espera revisión) → **aprobado** / **rechazado-devuelto** (con motivo/observaciones).

10.1 Inicio (`/`)

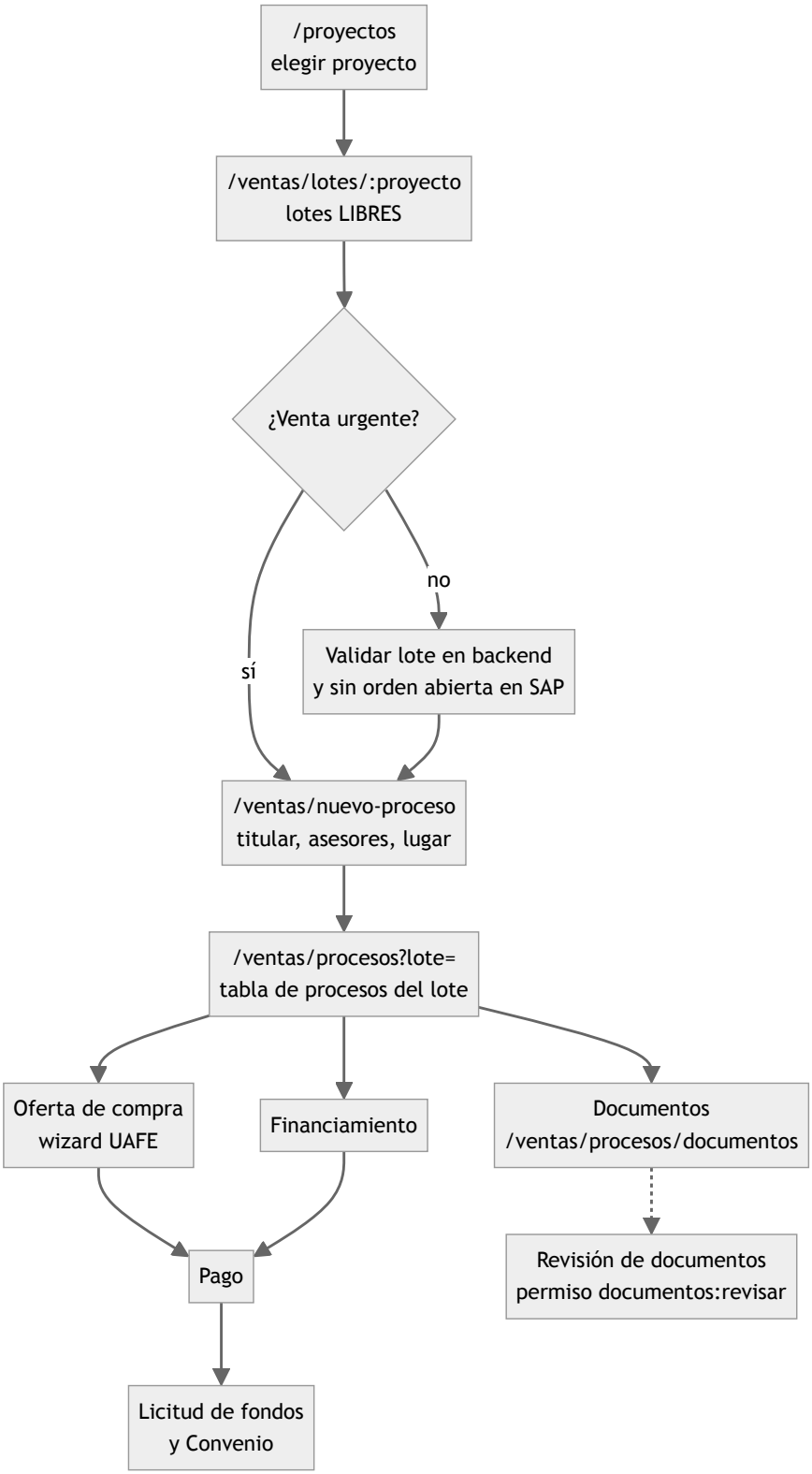
KPIs y gráfico de ventas por mes (`/dashboard/resumen`), lista de tareas personales (`/tareas`), mini-calendario con eventos (`/eventos`; crear/eliminar con `eventos:gestionar`), banner de bienvenida y entregas pendientes (solo si el usuario tiene algún permiso de entregas).

10.2 Ventas — Inventario

Ruta	Permiso	Función
`/ventas/inventario/catalogo`	`inventario:ver`	Catálogo de solo lectura con 3 vistas: **Simple** (grilla de lotes LIBRES, ficha detallada), **Planos** (imagen con marcadores), **Detallada** (tabla de precios/cuotas). Filtro de presupuesto mín/máx. Fallback offline vía IndexedDB.
`/ventas/inventario`	`inventario:gestion`	CRUD de lotes y proyectos, cambio de estado con motivo, **importación masiva** (CSV o pegado de Excel → vista previa validada por backend → importación todo-o-nada), ordenar proyectos, subir imagen/plano (PDF → PNG).
`/ventas/inventario/plano/[id]`	`proyectos:gestion`	Editor de mapeo: ubica cada lote en el plano como pin, rectángulo o polígono; zoom; lotes en convenio se pintan discretos con check.

Reglas de estado de lote: el backend puede devolver estados con espacios (`" LIBRE"`) → siempre `trim`. *Convenio*, *Vendido*, *Hipotecado* y *Facturado* (derivado de tener `numFac` y `anioFac`) cuentan como vendidos. Eliminar un lote es lógico (el backend nunca borra la fila). Las fechas `YYYY-MM-DD` se formatean a mano (sin `Date`) para evitar el corrimiento por zona horaria.

10.3 Ventas — Proceso de venta



Paso	Ruta	Reglas clave
Nuevo proceso	`/ventas/nuevo-proceso` (`ventas: nueva`)	Exige lote cargado, ≥1 asesor (el logueado preseleccionado), lugar de venta y datos completos del titular (nombre, **identificación válida**, estado civil, nacimiento, sexo, nacionalidad). La venta urgente omite validación de disponibilidad y SAP. Admite OCR de cédula/pasaporte (`/ventas/documento-identidad`).

Paso	Ruta	Reglas clave
Procesos del lote	`/ventas/procesos`	Puede haber varios procesos por lote mientras ninguno esté 100 % aprobado. Columna de documentos "x/y". Cada columna (oferta/financiamiento/pago) abre el formulario (pendiente) o el reporte (ingresado/aprobado/rechazado).
Documentos	`/ventas/procesos/documentos?proceso=`	Catálogo en `lib/const/documentos-proceso.ts` (cédula frontal/reverso, papeleta, servicio básico, SRI, buró, rol de pago *repetible*). Estados: subido / no aplica / pendiente / solicitado. Tipos personalizados renombrables. El revisor solicita y verifica en la misma pantalla.
Oferta de compra	`/ventas/oferta/new/[id]`	Wizard (~2 500 líneas): cliente (+ hasta 3 clientes adicionales), cónyuge (si casado), trabajo, datos financieros (`oferta_pld` , capacidad de ahorro calculada), **licitud de fondos / PEP** (formulario UAFE), asesor y referidos. Crear: `ventas:crear` ; reabrir: `ofertas:editar` . Valida identificación, correo y teléfono por campo. Muestra observaciones de devoluciones.
Revisión de oferta	`/ventas/oferta/reporte/[id]`	`ofertas:revisar` : aprobar o devolver marcando campos incorrectos con comentario.
Financiamiento	`/ventas/financiamiento/new/[id]`	Tipos: **Contado** (5 % desc. fijo, fila "Pago total"), **Crédito directo** (cronograma con interés, escritura en las últimas 6 cuotas), **Crédito BIESS** (3 % desc., fila "Desembolso BIESS"), **Mixto** (cronograma + desembolso BIESS extra). Alícuota = suma de componentes del proyecto según fecha de inicio. Promociones aplicables. El cronograma real lo calcula el backend; el front solo previsualiza. Editar: `financiamiento:editar` ; aprobar: `financiamiento:aprobar` .
Pago	`/ventas/pagos/new/[id]`	Solo con oferta y financiamiento aprobados. El valor = abono inicial del financiamiento (no lo escribe el asesor). Nace "ingresado". Reabrir: `pago:editar` ; aprobar/devolver: `pago:aprobar` .
Licitud	`/ventas/licitud/reporte/[id]`	`licitud:imprimir` . Formato A4 imprimible; % de participación repartido en partes iguales entre beneficiarios.
Convenio	`/ventas/convenio/reporte/[id]`	Plantilla (Configuración → Convenios) resuelta por el backend con los datos del proceso + bloque de firmas elegido en el momento; exporta a Word.
Impresión	`.../reporte/[id]/imprimir`	Versiones de impresión de oferta, financiamiento y pago.

Otras pantallas de ventas:

Ruta	Permiso	Función
`/ventas/mis-ventas`	`ventas:crear`	Ventas 100 % aprobadas del asesor logueado (el backend lo resuelve por sesión).
`/ventas/realizadas`	`ventas:reporte`	Ventas cerradas de toda la empresa (acceso a sus reportes una vez que el lote deja de estar LIBRE).

Ruta	Permiso	Función
`/ventas/reporte`	`ventas:reporte`	Reporte de ventas global con filtros.
`/ventas/revision-documentos`	`documentos:revisar`	Cola paginada de procesos pendientes de revisión.
`/ventas/urgentes`	`venta:urgente`	CRUD de lotes marcados como venta urgente.

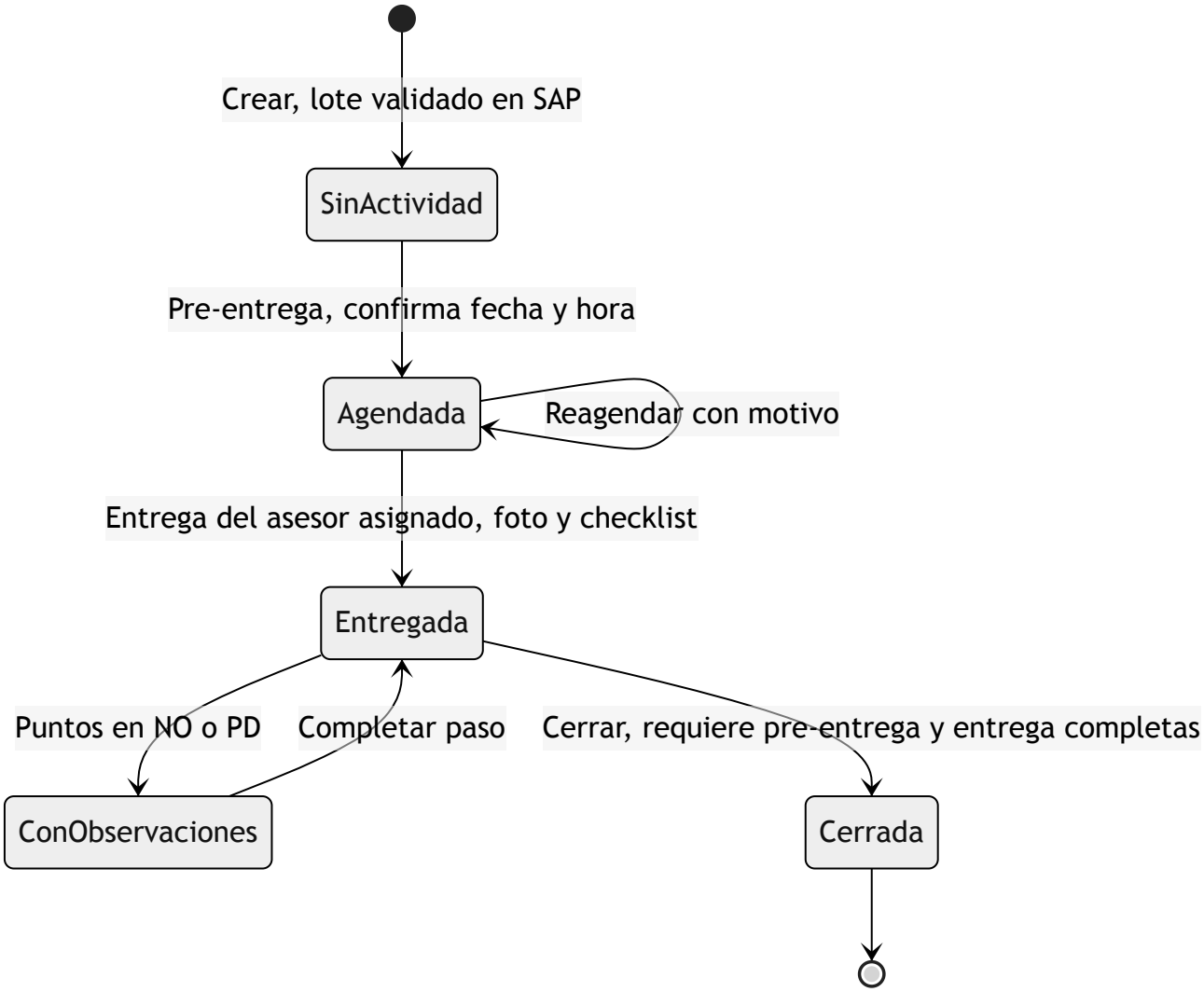
10.4 Gestión — Cotizaciones (`/gestion/cotizaciones`)

Solo requiere sesión. Lista "Mis cotizaciones" y, con `cotizaciones:ver_todas`, "Todas". Flujo: elegir proyecto → lote libre → datos del cliente → simulador (mismas reglas de tipos y descuentos que Financiamiento, con promociones) → reporte e impresión. No tiene aprobación: "ingresado" es el estado final. Usa rutas propias del cotizador (no las de inventario) para que un colaborador sin permisos de ventas no reciba 403.

10.5 Gestión — Refinanciamiento (`/gestion/refinanciamiento`, `refinanciamiento:crear`)

Elegir proyecto → lotes vendidos → nuevo refinanciamiento sobre el proceso. Diferencias con Financiamiento: tipo adicional **"Crédito BIESS (Sin Interés)"** (mismas reglas que BIESS), tasa anual **editable** (default = tasa fija), abono a capital, abonos realizados, historial por proceso (`/historial/[procesoId]`), aprobar/rechazar (`refinanciamiento:aprobar`). Para ventas históricas sin `sale_process`, `/registrar-venta/[codigo]` crea el proceso mínimo (nombre + cédula).

10.6 Entregas (`/entregas`) — permisos explícitos



Permisos por transición: crear `entregas:crear`, pre-entrega `entregas:pre_entrega`, reagendar `entregas:reagendar`, entrega `entregas:entregar`, completar paso y cerrar `entregas:cerrar`.

- Crear: busca la orden abierta del lote en SAP; lote y cliente son de solo lectura; un lote solo puede tener un proceso (409). Huerto/tanque biodigestor marcados "SI" si estaban en la oferta.
- Checklist con valores SI / NO / PD; en la entrega se exige el "por qué" cuando un punto difiere de la pre-entrega o está en PD.
- Foto re-codificada a JPEG ≤ 1600 px antes de subir.
- Advertencia de conflictos de agenda del asesor (`/entregas/conflictos`).
- Pestañas (filtros no excluyentes): agendadas, sin actividad, con observaciones, etc. Fila amarilla: 2 días después de la fecha sin entrega.
- Reporte PDF/CSV de lo filtrado (`entregas:reporte`), generado en el navegador.

10.7 Tesorería — Depósitos (`/tesoreria/depositos`, `depositos:ver`)

- Los clientes suben comprobantes desde la app; llegan en **tiempo real** por socket (`use-depositos-socket.ts`, sala `empresa_<id>`).
- Al abrir los comprobantes de un depósito pendiente, el backend lo pasa a **"En Revisión"** y notifica a la app del cliente.

- Acciones (requieren `depositos:gestionar` en UI): aprobar con recibo de caja `RC-` + hasta 6 dígitos y adjuntos (≤ 3 archivos por request), devolver con motivo, aprobar un lote devuelto asociando el recibo `REC-YYYYMMDD-XXXX`.
- Estado de la tarjeta derivado de sus lotes: Aprobado/Devuelto solo si **todos** lo están.
- Un comprobante puede cubrir varios lotes (`lotesSoloEnResumen`).
- Lista de presencia: quién más está revisando.
- Lógica pura en `depositos-utils.ts` (cubierta por tests).
- `/tesoreria/retiros`: **maqueta** con datos de ejemplo.

10.8 RRHH (`/rrhh`)

Pestañas (`?tab=` permite abrir una directa desde una notificación). Cada colaborador crea y ve lo suyo; `rrhh:gestionar` revisa todo, aprueba/rechaza y administra.

Pestaña	Reglas
Vacaciones	Periodos disponibles definidos por RRHH; una solicitud vigente bloquea otra. Vacaciones colectivas de diciembre (inicio + 8 días). Reporte PDF. Saldo = disponibles (15 + 1 cada 5 años) + aumentados – descontados (cálculo del backend).
Permisos	Fecha posterior a hoy. Tipos con descuento: al aprobar se elige la fuente (vacaciones / del día) viendo el saldo. Rechazo con motivo. PDF.
Préstamos	Fecha posterior a hoy. Aprobar / rechazar. Carta PDF.
Usuarios	Solo `rrhh:gestionar`: colaboradores y ajuste de saldo.
Documentos	General (todos), Mis documentos (expediente propio) y Expedientes (gestión). Bucket privado → URL firmada temporal; visor interno para PDF/imagen.

10.9 Reportes (`/reportes`)

- Catálogo programado en `lib/reportes/catalogo.ts`: `ventas`, `administrativo`, `depositos`, cada uno con permiso `reportes:<id>` **explícito** (el Administrador no los ve si no se los asignan).
- **Constructor** (`/reportes/administrar`, `reportes:administrar`): definiciones dinámicas sobre datasets del backend; cada una se expone como `d:<slug>` con permiso `reportes:<slug>`, filtrado por el backend.
- Enlace directo `?r=<id>` (solo si está permitido).

10.10 App móvil de clientes (`/app/*`, sección del menú solo Administrador)

Ruta	Acceso	Estado
`/app/usuarios`	Administrador	**Datos de ejemplo**. Endpoints a conectar documentados en `app/usuarios/datos.ts`.
`/app/notificaciones`	Administrador	**Datos de ejemplo**. Envío individual / global / segmentado (mora, cédulas, lotes, prefijos), programación, vista previa de pantalla bloqueada.
`/app/diagnostico`	`app:diagnostico`	**Conectado** a `/telemetry/*`: resumen, tendencias 24h/7d, grupos de error, eventos, sesiones, vista por usuario (máx. 7 días / 500 registros).

`/notificaciones` y `/notificaciones/historial` son una versión anterior del envío push, conectada a `notificacionesApi`, fuera del menú.

10.11 Administración

Ruta	Acceso	Función
`/admin/usuarios`	Administrador o `admin:user`	Usuarios de Better Auth: crear, editar, rol admin, banear, eliminar, cambiar contraseña (con `REGLAS_PASSWORD`), roles relacionales, overrides de permisos, sesiones activas (revocar).
`/admin/roles`	`admin:user`	Matriz recurso × acción; crear/renombrar/eliminar roles y permisos; cambios optimistas + recompilación.
`/admin/auditoria`	`auditoria:ver`	Visor de eventos (filtros con debounce, detalle). Sus propios clics no se auditan.
`/configuracion`	Administrador	Índice de secciones de configuración.
`/configuracion/convenios`	`convenios:gestionar`	Plantillas con editor Tiptap, variables `{{clave}}` (Handlebars) ligadas a entidad/campo o de "selección manual", asignación a proyecto/lote, vista previa; representantes de la empresa. Edición solo en escritorio (`SoloEscritorio`).
`/configuracion/promociones`	`promociones:gestionar`	CRUD de promociones (no se puede eliminar una ya aplicada).

10.12 Otras pantallas

- `/gestion/encuesta`: encuesta de satisfacción — **maqueta** sin backend.
- `/ventas/financiamiento` (índice): **maqueta** de tabla de pagos con datos fijos.
- `/sin-acceso`: muestra la sección denegada y el rol actual.

---

11. Validaciones

11.1 Principio

Las reglas viven **una sola vez** en `lib/validations/` y se reutilizan en formularios y servidor. Los validadores "de campo" devuelven `string | null` (mensaje o válido) y **no marcan error mientras el valor está incompleto**.

11.2 Catálogo

Archivo	Regla
`auth.ts`	`loginSchema` : email válido + password no vacío (no se valida composición en login). `passwordSchema` / `REGLAS_PASSWORD` : ≥ 8 caracteres, 1 mayúscula, 1 número (al definir contraseña desde admin).
`identificacion.ts`	Tipo inferido por forma: **10 dígitos** → cédula ecuatoriana (provincia 01–24, tercer dígito ≤ 5, dígito verificador módulo 10); **13 dígitos** → RUC (solo longitud); con letras → **pasaporte** alfanumérico sin espacios.
`email.ts`	`algo@algo.algo` sin espacios.
`telefono.ts`	Solo dígitos (el prefijo de país va en un selector aparte y se concatena al guardar).
`convenios.ts`	`convenioSchema` (nombre, contenido, variables con clave única). `variableDefinicionSchema` : clave `^[a-z0-9_]+$` , entidad ∈ {proyecto, inventario, titular, conyuge, apoderado, financiamiento, oferta, proceso, sistema, seleccion}; `seleccion` exige ≥ 2 opciones sin valores repetidos, el resto exige `campo` . Debe coincidir 1:1 con el DTO de anconAPI.
`documentos-proceso.ts`	Estado de cada documento, resumen x/y (excluye repetibles), `slugificarTipo` (sin tildes, `[a-z0-9_]` , máx. 50), `etiquetaDesdeTipo` .
`financiamiento.ts`	Schema de ejemplo (monto ≤ 10 M, plazo 1–360 meses).

11.3 Validaciones de negocio en pantallas (resumen)

- Tesorería: formato `RC-######` y `REC-YYYYMMDD-XXXX` (`depositos-utils.ts`).
- RRHH: fechas de permisos/préstamos posteriores a hoy (igual que el backend).
- Oferta: bloqueo de avance de paso si hay identificación/correo/teléfono inválido.
- Pago: habilitado solo con oferta y financiamiento aprobados (también por URL directa).
- Importación de lotes: validación en backend sin guardar (vista previa) y todo-o-nada.
- Imágenes de entregas: re-codificación a JPEG.

> **Recordatorio:** toda validación de cliente es UX. anconAPI debe revalidar cada regla.

12. Componentes compartidos y UI

12.1 Sistema de diseño

- Fuentes: **Plus Jakarta Sans** (texto) y **JetBrains Mono** (códigos).
- Color de marca: `--primary #f5891f` → `--primary-to #e1251b` (degradado `bg-brand-gradient`).
- Estados semánticos (claro/oscuro): `status-disponible` (verde), `status-reservado` (ámbar), `status-bloqueado` (rojo), `status-hipotecado` (azul), cada uno con `-fg`, `-bg`, `-border`.
- Radio base `0.625rem`. Tokens en `app/globals.css`.
- **Responsive / móvil:** sidebar colapsable con drawer en móvil, `env(safe-area-inset-*)`, pull-to-refresh, AppShell `fixed inset-0`, `SoloEscritorio` para editores complejos.

12.2 `components/ui` (shadcn)

accordion, avatar, badge, button, card, checkbox, dropdown-menu, form, input, label, popover, progress, select, separator, sheet, skeleton, tabs, textarea, tooltip.

> Los generó el CLI asumiendo React 19. `Button` e `Input` se ajustaron con `React.forwardRef` para que `asChild` / `Slot` funcione en React 18. Revisar lo mismo al agregar primitivos con `npx shadcn add`.

12.3 `components/shared` (principales)

Grupo	Componentes
Shell	`AppShell`, `SidebarNav`, `SidebarContext`, `AppTopbar`, `DashboardTopbar`, `PageHeader`, `ProjectTabsBar`, `NavigationProgress`, `PullToRefresh`
Sesión/seguridad	`SessionProvider` / `useSession`, `AuthProvider`, `Can` / `useCan`, `SesionGuardian`, `LoginForm`, `VerificacionForm`
Tiempo real/auditoría	`PresenciaProvider`, `AuditoriaProvider`, `NotificacionesBell`, `NotificacionesInternasBell`
Formularios	`TextField`, `SelectField` / `StaticField`, `CampoSelect`, `ComboboxField`, `MultiSelectField`, `DatePickerField`, `TimePickerField`, `PhoneField`, `ComplianceFormGrid`, `DocumentoIdentidadUploader`, `CapturaDocumentoDialog`, `OpcionRevision`, `MotivoRechazo`, `RatingScale`
Datos/feedback	`StatusBadge`, `KpiCard`, `SalesBarChart`, `MiniCalendar`, `TaskChecklist`, `Pagination`, `FilterBar`, `PillTabs`, `Stepper`, `EmptyState`, `AlertBanner`, `ConfirmDialog`, `RowActions`, `Spinner`, `ToastProvider`, `VistaDocumento`, `CodeChip`, `ProjectCard`, `WelcomeBanner`
Catálogos	`select-options.ts` (estados civiles, tipos de financiamiento, etc.)

12.4 Navegación

Usar `import { useRouter } from "@/lib/navigation"` (no `next/navigation`): dispara la barra de progreso antes de navegar. Los demás exports de `next/navigation` se re-exportan igual.

13. Infraestructura transversal (auditoría, presencia, PWA, offline)

13.1 Auditoría de interacción (`AuditoriaProvider` + `lib/auditoria/tracker.ts`)

- Captura clics en elementos accionables (`a`, `button`, roles ARIA, `[data-audit]`) y cambios de ruta. **Nunca** captura valores de inputs; solo etiqueta, id y pathname (sin query).
- Cola en memoria (máx. 500), envío en lotes de 50 cada 4 s a `POST /auditoria/eventos`; al ocultar/cerrar la pestaña usa `keepalive`. Si falla, reintenta en el próximo ciclo.
- `registrarAccion("DEPOSITO_APROBADO", etiqueta, detalle)` para acciones de negocio explícitas tras una operación exitosa.
- `data-audit="..."` define la etiqueta; `data-audit-ignore` excluye una zona.

13.2 Presencia en tiempo real (`PresenciaProvider`)

Un único socket por pestaña a `{API}/aplication` (solo transporte websocket). Sala `empresa_inmoconstruccciones` (la triple "c" es el id real del backend). Publica el módulo (ruta) actual del usuario y expone `{ socket, conectado, enLinea }` para que otras pantallas (p. ej. depósitos) escuchen eventos propios.

13.3 PWA

- `app/manifest.ts` → instalable (standalone, portrait, ícono maskable).
- `public/sw.js` registrado por `PwaRegister` **solo en producción** y con HTTPS/localhost; `public/offline.html` como página sin conexión.
- Metadatos Apple (`appleWebApp`) y `viewportFit: cover`.

13.4 Offline de planos

`lib/plano-cache.ts` guarda imagen (Blob) + lotes ubicados en IndexedDB cada vez que se abre un plano con conexión; el catálogo lo usa como respaldo si la carga online falla. Errores de IndexedDB se ignoran en silencio.

---

14. Generación de documentos (PDF / Word / CSV)

Salida	Librería	Dónde	Notas
Reporte de entregas (PDF/CSV)	jsPDF (import dinámico)	`lib/entregas-reporte.ts`	CSV con `;` + BOM UTF-8 para Excel en español
Reportes RRHH (PDF)	jsPDF (import dinámico)	`lib/rrhh-reportes-pdf.ts`	Escala A4 827×1169 "px de diseño"; fechas armadas a mano
Convenio (Word)	html-to-docx	`convenio-reporte-content.tsx`	Anchos de tabla en **px**, nunca `%` (rompe el XML); HTML saneado con `limpiar-html-word.ts`
Plantilla de importación (CSV)	—	`lib/importar-lotes.ts`	`;` + BOM
Plano PDF → PNG	pdfjs-dist (import dinámico)	`lib/pdf-a-imagen.ts`	Escala 2×
Impresión de reportes	`window.print()` + rutas `/imprimir`	ventas	Se oculta el sidebar en impresión (`data-app-sidebar`)

`next.config.mjs` define `fs`, `path` y `encoding` como `false` en el bundle de cliente para que html-to-docx compile.

15. Testing y calidad

- **Vitest** (`jsdom`, globals, `tests/setup.ts` con jest-dom). Alias `@` igual que TS.
- Estructura de `tests/` espeja la del código.

Suite	Cubre
`tests/lib/auth/rbac.test.ts`	`can()`: match exacto, denegación, separación de funciones
`tests/lib/reportes/catalogo.test.ts`	Reportes explícitos (sin bypass admin, `reportes:all`)
`tests/lib/auditoria/tracker.test.ts`	Cola, lotes de 50, reintento, keepalive
`tests/app/tesoreria/depositos-utils.test.ts`	Estado derivado, formatos RC/REC, resumen
`tests/lib/validations/auth.test.ts`	`loginSchema`

Estado al 21/10/2026: 33 tests pasan y **3 fallan** en `auth.test.ts`: esperan que `loginSchema` rechace passwords débiles, pero el schema se relajó a "no vacío" (correcto para el login con Better Auth). Hay que mover esos casos a `passwordSchema`.

- **ESLint:** `next/core-web-vitals` + `next/typescript` + `prettier`; `@typescript-eslint/no-explicit-any: error`.
- **Prettier:** sin `;`, comillas dobles, `trailingComma: es5`, ancho 90, orden de clases Tailwind.

16. Convenciones de código

- **Idioma:** infraestructura en inglés (`lib`, `session`, `password`); dominio de negocio en español (`Rol`, `Permiso`, `lote`, `financiamiento`, `requirePermiso`). Las tablas legacy conservan su nombre original.
- **Archivos:** kebab-case. Pantallas: `page.tsx` (servidor) + `<nombre>-content.tsx` (cliente); paneles laterales `*-sheet.tsx`; helpers puros en `*-utils.ts`, `datos.ts`, `formato.ts`.
- **Comentarios:** explican el *por qué* (decisiones, restricciones del backend, bugs evitados). Mantener esa densidad al modificar código.
- **Constantes espejadas del backend** (deben cambiarse en ambos repos): tasa de interés fija (`amortizacion.util.ts`), descuentos 5 % Contado / 3 % BIESS, catálogo de documentos de proceso (`documentos-catalogo.const.ts`), días de vacaciones colectivas, tipos de permiso con descuento, DTO de variables de convenio.
- **Server-only:** todo lo que use Prisma o cookies importa `"server-only"`.
- **Fechas:** formatear `YYYY-MM-DD` manualmente; `toLocaleString("es-EC")` para fechas con hora.
- **Imports pesados** (jspdf, pdfjs, html-to-docx) siempre dinámicos dentro de la función.

17. Guías: cómo extender el sistema

17.1 Agregar una pantalla nueva protegida

1. Crear `app/(dashboard)/<ruta>/page.tsx`:

   ```
   export default async function Page() {
     await requirePermiso("modulo:accion", "Nombre visible")
     return <MiContent />
   }
   ```

   Usar `requirePermisoExplicito` si el backend valida sin bypass de admin.
2. Crear `<mi>-content.tsx` con `"use client"` y consumir datos desde `lib/api`.
3. Agregar la entrada en `NAV_ITEMS` de `components/shared/sidebar-nav.tsx` con el mismo permiso.
4. Crear el permiso (Admin → Roles, o `upsert` en `prisma/seed.ts`) y asignarlo.
5. Asegurar que el endpoint de anconAPI valide el mismo permiso.

17.2 Agregar un endpoint

- Mismo prefijo existente → agregar función al archivo de `lib/api` correspondiente con su comentario `// VERBO /ruta` y tipos copiados de la respuesta real.
- Prefijo nuevo → nuevo archivo siguiendo la plantilla de `lib/api/README.md`.

17.3 Agregar un permiso

1. `upsert` en `prisma/seed.ts` (o crearlo desde `/admin/roles`).
2. Usarlo en guard + menú + `<Can>`.
3. Validarlo en el controller de anconAPI.

17.4 Agregar un reporte programado

1. Entrada en `lib/reportes/catalogo.ts` (`id`, `titulo`, `descripcion`, `permiso`).
2. Componente registrado en `app/(dashboard)/reportes/reportes-content.tsx`.
3. Crear `reportes:<id>` (anconAPI `add_reportes_permisos.js`) y asignarlo.

17.5 Agregar un tipo de documento de proceso

Entrada en `lib/const/documentos-proceso.ts` **y** en `anconAPI/apps/intranet/ventas/const/documentos-catalogo.const.ts`.

17.6 Agregar una tabla propia

Modelo con `@@map("<nombre>_app")` → `migrate diff --script` → revisar → aplicar a mano → `db:generate` (§8.3).

---

18. Despliegue

Checklist:

- [ ] Variables de entorno de producción (`DATABASE_URL`, `AUTH_JWT_SECRET` ≥ 32 aleatorio, `NEXT_PUBLIC_API_URL` con HTTPS).
- [ ] `NEXT_PUBLIC_API_URL` se embebe en el bundle **en build**: reconstruir al cambiarla.
- [ ] `npm run build` sin errores de tipos ni lint; `npm run test` en verde.
- [ ] HTTPS obligatorio (cookie `Secure`, service worker, cámara para documentos).
- [ ] CORS de anconAPI con el origen de la intranet y `credentials: true`.
- [ ] Better Auth fijado en **1.6.11** en anconAPI (otra versión rompe el login con "user not found").
- [ ] `JWT_SECRET` configurado en anconAPI y permisos `depositos:*` creados (ver informes).
- [ ] Permiso `app:diagnostico` y tabla de telemetría creados a mano en la BD.
- [ ] Ejecutar `npx tsx prisma/migrar-catalogo-permisos.ts` si hay usuarios con permisos solo en `user.permissions`.
- [ ] Rate limiter compartido si se escala a varias instancias.
- [ ] Revisar `informes/informe-seguridad-criticos-2026-10-15.pdf`.

19. Estado actual, deuda técnica y riesgos

#	Tema	Detalle	Recomendación
1	Control de versiones	Casi todo el código está sin commitear (solo existe el commit del scaffold).	Commitear por módulos cuanto antes.
2	Tests rotos	3 casos de `loginSchema` fallan (ver §15).	Moverlos a `passwordSchema`.
3	Doble sistema de sesión	Sesión legacy (JWT + `sesiones_app` + `credenciales_acceso`) sigue en el código pero el login usa Better Auth.	Decidir retiro de la sesión legacy y del rate limiter asociado.
4	README desactualizado	Menciona `/solicitar-acceso` y `establecer-password` (no existen) y el login propio.	Actualizar o enlazar este documento.
5	`.env.example`	Ejemplo de PostgreSQL, el proveedor real es MySQL.	Corregir.
6	Filtro de rol en middleware	Solo aplica a la sesión legacy; con Better Auth no filtra `/admin`.	Aceptable (las páginas verifican), pero documentado.
7	`user.permissions`	Se sigue recompilando aunque ya no autoriza.	Eliminar la escritura cuando anconAPI no lo lea.
8	Pantallas maqueta	`/tesoreria/retiros`, `/gestion/encuesta`, `/ventas/financiamiento` (índice), `/app/usuarios`, `/app/notificaciones`.	Conectar (endpoints documentados en `datos.ts`) o retirar.
9	Endpoints faltantes	`GET /ventas/procesos/personas` y precarga de clientes extra no existen en el backend; el panel "Editar proceso" tiene un comentario de conexión pendiente (verificar contra `actualizarProceso`).	Implementar en anconAPI.
10	Duplicación front/back	Constantes financieras y catálogos repetidos en ambos repos.	Paquete compartido o endpoint de configuración.
11	Archivos gigantes	Oferta (~2 500 líneas), financiamiento/cotización/refinanciamiento (~3 400–3 600 c/u) con lógica muy similar.	Extraer motor de tabla de pagos común.
12	Tres lockfiles	`package-lock.json`, `pnpm-lock.yaml`, `bun.lock`.	Elegir un gestor y borrar los demás.
13	Cookie Better Auth	`useSecureCookies: false` en anconAPI.	Activar en producción (el nombre pasa a `__Secure-better-auth.session_token`: actualizar `BETTER_AUTH_COOKIE_NAME`).
14	Rutas públicas en anconAPI	Varias rutas de app móvil son públicas (informe de seguridad).	Protegerlas antes de conectar `/app/*`.
15	Cobertura de tests	Solo lógica pura; sin tests de componentes ni de flujos.	Priorizar cálculos financieros y guards.

20. Glosario

Término	Significado
Proceso de venta	`sale_process`: venta de un lote a un titular; agrupa oferta, financiamiento, pago, documentos.
Titular	Cliente principal del proceso (`process_users`, rol titular).
Oferta de compra	Formulario de datos del cliente + PLD/UAFE; requiere aprobación.
Licitud de fondos / PEP	Declaración de origen de fondos y Persona Expuesta Políticamente (normativa UAFE).
Convenio	Contrato generado desde una plantilla con variables.
BIESS	Banco del IESS; crédito hipotecario externo.
Alícuota	Cuota de mantenimiento del proyecto, compuesta por componentes con fecha de inicio.
Escritura	Gastos de escrituración, cobrados en cuotas o en la fila de cierre según el tipo.
Venta urgente	Lote habilitado para venderse sin validación de disponibilidad ni orden SAP.
Pre-entrega / Entrega / Cierre	Etapas de la entrega física del lote al cliente.
PD	"Pendiente" en el checklist de entregas.
RC / REC	Número de recibo de caja / código de recibo de tesorería.
Override	Permiso concedido o denegado a un usuario puntual, por encima de su rol.
Permiso explícito	Permiso que no se hereda del bypass de Administrador.
anconAPI	Backend NestJS que expone la API, Better Auth y el WebSocket.